

GRPO

Group Relative Policy Optimization

东川路第一可爱猫猫虫

主要内容

- 从PPO到GRPO的改进
- PPO存在的问题
- GRPO的损失函数
- GRPO的完整流程
- GRPO的KL散度计算

Group Relative Policy Optimization



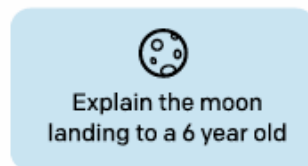
DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

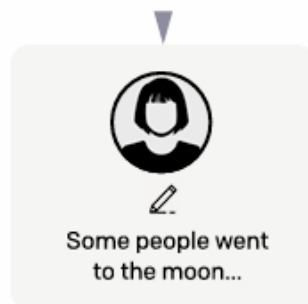
Step 1

**Collect demonstration data,
and train a supervised policy.**

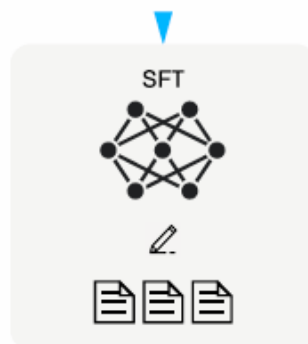
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



This data is used
to fine-tune GPT-3
with supervised
learning.

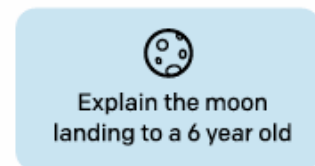


π_{ref} 参考模型

Step 2

**Collect comparison data,
and train a reward model.**

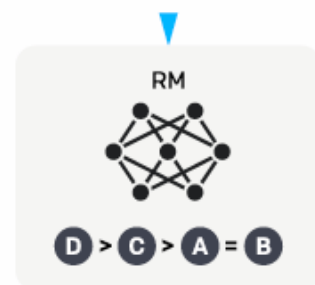
A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



r_ϕ 奖励模型

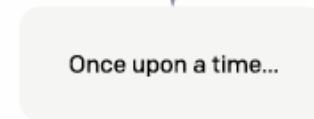
Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

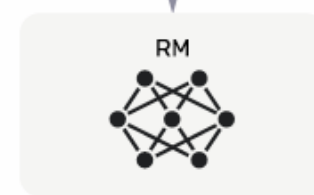
A new prompt
is sampled from
the dataset.



The policy
generates
an output.



The reward model
calculates a
reward for
the output.



The reward is
used to update
the policy
using PPO.



LLM是自回归条件分布

- 大语言模型生成回答的过程

next token prediction

- 给定一个prompt q

模型按token逐步生成序列 $o = [o_1, o_2, \dots, o_T]$

- 第 t 步生成token o_t 时

模型看到的是 q 和 $o_{<t} = (o_1, \dots, o_{t-1})$

每个token o_t 的生成概率是 $\pi_{\theta}(o_t|q, o_{<t})$

- 自回归：条件对象是它自己序列里的过去

PPO 多轮循环 更新模型

- PPO第一步：用旧模型采样得到序列

- 每一轮的更新里

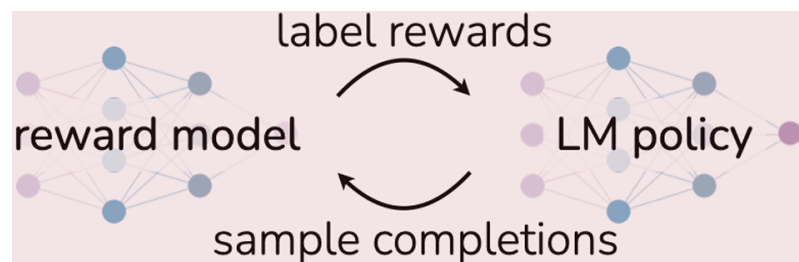
π_{θ} 是本轮正在更新的模型， $\pi_{\theta_{old}}$ 是上一轮更新后的模型

- 从问题数据集 $P(Q)$ 中采样一个问题 q

用旧模型 $\pi_{\theta_{old}}$ 生成 $o = [o_1, o_2, \dots, o_T]$

其中每个token o_t 的生成概率是 $\pi_{\theta_{old}}(o_t | q, o_{<t})$

- 这样就得到了序列 (q, o)



PPO第二步

- 为序列 (q, o) 里的每个token计算奖励

$$r_t = r_\phi(q, o_{\leq t}) - \beta \log \frac{\pi_{\theta_{\text{old}}}(o_t|q, o_{<t})}{\pi_{\text{ref}}(o_t|q, o_{<t})}$$

$\frac{\pi_{\theta_{\text{old}}}(o_t|q, o_{<t})}{\pi_{\text{ref}}(o_t|q, o_{<t})}$ 是重要性权重

$r_\phi(q, o_{\leq t})$ 是奖励模型打的分数

- 通常只有最后一个token会被奖励模型赋予奖励分数
- 广义优势估计

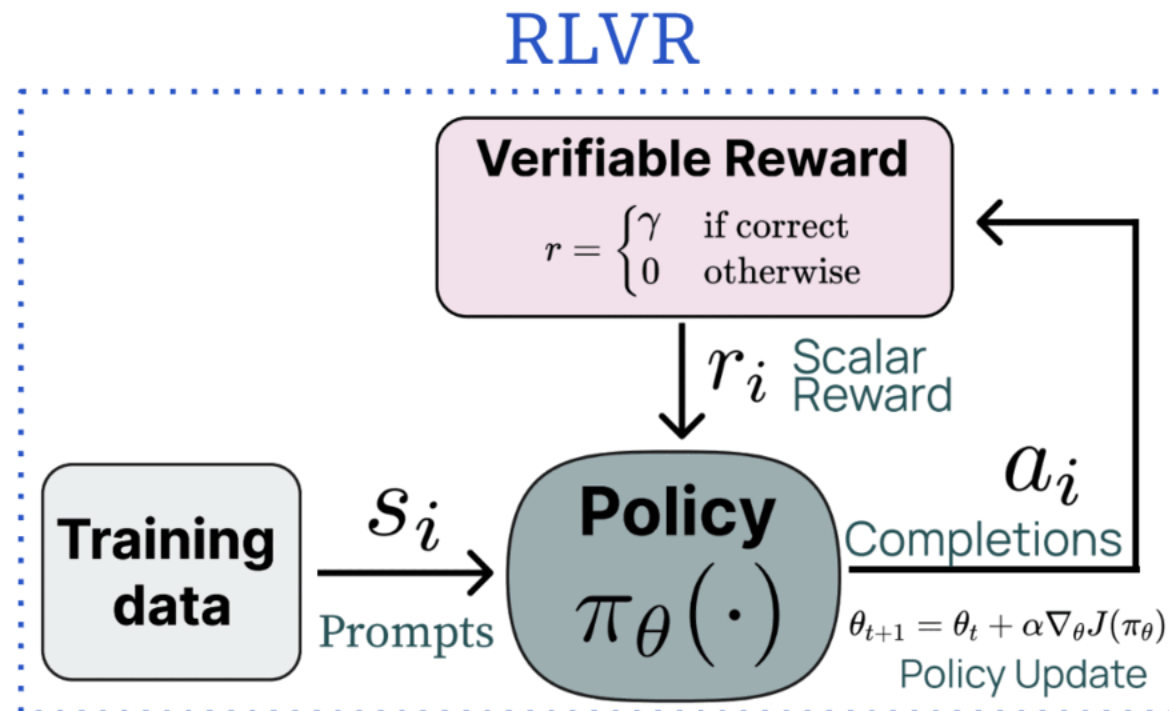
由 $r_{\geq t}$ 和 V_ψ 计算 A_t

PPO第三步

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_t \right]$$

- 最大化目标函数来更新模型 π_{θ}
- 最小化价值估计的误差来更新价值模型 V_{ψ}
- 本轮PPO更新完成
- 下一步将旧模型 $\pi_{\theta_{old}}$ 更新为刚刚得到的当前模型 π_{θ}
进入下一轮的采样与更新的循环

- Start with a dataset of diverse prompts
- Iteratively do the following:
 - Sample a mini-batch of prompts for training
 - Generate on-policy completions for the prompts
 - Compute the reward for each completion
 - Use rewards to compute a policy update



RLHF

Step 3

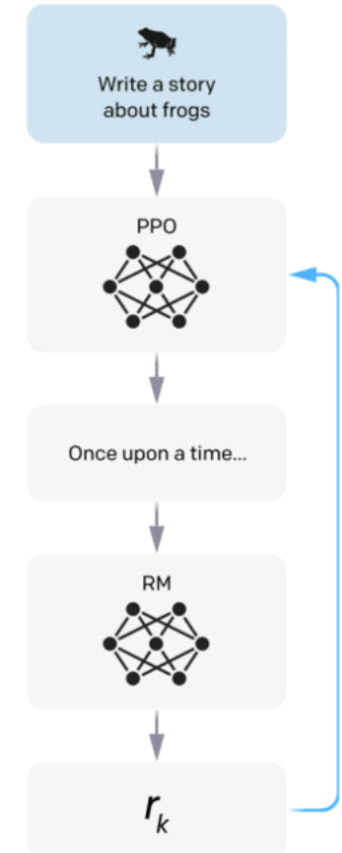
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

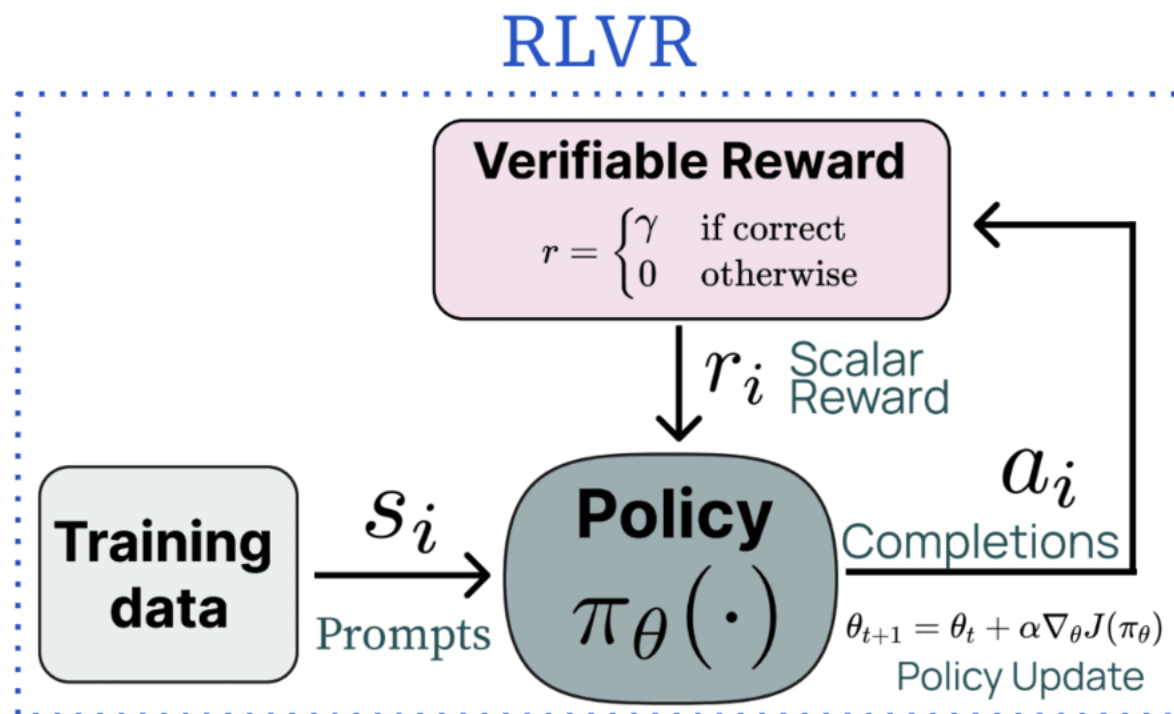
The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



- Start with a dataset of diverse prompts
- Iteratively do the following:
 - Sample a mini-batch of prompts for training
 - Generate on-policy completions for the prompts
 - Compute the reward for each completion
 - Use rewards to compute a policy update



RLHF

Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.



Once upon a time...

The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



PPO的两项问题

As the **value function** employed in PPO is typically another model of comparable size as the policy model, it brings a substantial **memory and computational burden**. Additionally, during RL training, the value function is treated as a baseline in the calculation of the advantage for variance reduction. While in the LLM context, usually **only the last token** **is assigned a reward score by the reward model**, which may complicate the training of a value function that is

- critic model
- 通常只有最后一个token会被奖励模型赋予奖励分数
 我们需要为每个token估计value，计算优势函数
 即：把序列末尾的奖励信号
 精准归因到前面每一个 token 的生成决策上
- critic model 又大又难学

Group Relative Policy Optimization



DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

- GRPO对PPO做的改进:
 - 去掉critic, 不再用GAE来计算优势
 - 使用对同一问题采样得到的多个输出的**平均奖励**作为 baseline
 - KL 散度直接加入到 loss 中
 - 而不是加到奖励里面作为惩罚项

GRPO的损失函数

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_t \right]$$

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

GRPO的完整循环流程

- 从prompt dataset里采样一组prompt

$$D_b = \{q_1, q_2, \dots, q_B\}$$

- 对每一个prompt采样G个回答

$$\{o_1, o_2, \dots, o_G\}$$

$$\{o_i\}_{i=1}^G \sim \pi_{\theta}^{old}(\cdot | q)$$

每个 o_i 是token序列 $o_i = [o_{i,1}, o_{i,2}, \dots, o_{i,|o_i|}]$

- 奖励模型对每个 o_i 打分

$$\{r_1, r_2, \dots, r_G\}$$

GRPO的完整循环流程

- A的计算 组内相对优势估计

计算平均奖励 $\bar{r} = \frac{1}{G} \sum_{i=1}^G r_i$

计算奖励的标准差 $\sigma_r = \sqrt{\frac{1}{G} \sum_{i=1}^G (r_i - \bar{r})^2}$

计算优势 $A_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}$ (结果监督)

- 另一种组内相对优势估计 过程监督

对每个输出的每个步骤打分

$$R = \{\{r_{index(1)}^1, \dots, r_{index(K_1)}^1\}, \dots, \{r_{index(1)}^G, \dots, r_{index(K_G)}^G\}\}$$

每个 token 的优势为从该 token 之后所有步骤的标准化奖励之和

GRPO的完整循环流程

- 计算概率比

$$\text{ratio}_{i,t} = \frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})}$$

- 计算裁剪目标

$$g(\epsilon, A_i) = \text{clip}(\text{ratio}_{i,t}, 1 - \epsilon, 1 + \epsilon) \cdot A_i$$

- 对每个token $o_{i,t}$ 计算损失

$$L_{i,t} = \min(\text{ratio}_{i,t} \cdot A_i, g(\epsilon, A_i))$$

GRPO的完整循环流程

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

KL散度近似为

$$\mathbb{D}_{\text{KL}}[\pi_{\theta} || \pi_{ref}] = \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - 1$$

Approximating KL Divergence

Posted on 2020/03/07

- k1

$$k_1 = r \log \pi - \log \pi_{ref}$$

logprob - ref_logprob

无偏，求期望后等于真实KL散度
方差大

Approximating KL Divergence

Posted on 2020/03/07

- k2

$$k_2 = \frac{1}{2} r^2$$

`0.5 * (logprob - ref_logprob).square()`

数值上有偏，方差较低

Approximating KL Divergence

Posted on 2020/03/07

- k3

$$k_3 = e^{-r} + r - 1$$

数值上无偏，且方差极小

- 因此Schulman推荐k3估计器
- 因此GRPO原论文里用的就是k3估计器

$$k_3(x) = \frac{\pi_{\text{ref}}(x)}{\pi_{\theta}(x)} - 1 - \log \frac{\pi_{\text{ref}}(x)}{\pi_{\theta}(x)}$$

另外一个好处是k3恒为正

GRPO时代 KL散度被加到loss里

- 不仅需要关注KL散度估计器在数值上的准确性
还需要关注估计器的梯度是否足够准确
- k3估计器的梯度

$$k_3(x) = \frac{\pi_{\text{ref}}(x)}{\pi_{\theta}(x)} - 1 - \log \frac{\pi_{\text{ref}}(x)}{\pi_{\theta}(x)}$$

$$\nabla_{\theta} \left(\frac{\pi_{\text{ref}}(x)}{\pi_{\theta}(x)} \right) = \pi_{\text{ref}}(x) \cdot (-1) \pi_{\theta}(x)^{-2} \nabla_{\theta} \pi_{\theta}(x) = -\frac{\pi_{\text{ref}}(x)}{\pi_{\theta}(x)} \nabla_{\theta} \log \pi_{\theta}(x)$$

$$\nabla_{\theta} (-\log \pi_{\text{ref}}(x) + \log \pi_{\theta}(x)) = \nabla_{\theta} \log \pi_{\theta}(x)$$

$$\nabla_{\theta} k_3(x) = -\frac{\pi_{\text{ref}}(x)}{\pi_{\theta}(x)} \nabla_{\theta} \log \pi_{\theta}(x) + \nabla_{\theta} \log \pi_{\theta}(x)$$

$$\mathbb{E}_{\pi_{\theta}(\mathbb{X})}[\nabla_{\theta} k_3(x)] = \mathbb{E}_{\pi_{\theta}(\mathbb{X})} \left[-\frac{\pi_{\text{ref}}(x)}{\pi_{\theta}(x)} \nabla_{\theta} \log \pi_{\theta}(x) \right]$$

- $\nabla_{\theta} \text{KL}(\pi_{\theta} || \pi_{\text{ref}}) = \mathbb{E}_{\pi_{\theta}(\mathbb{X})} \left[-\log \frac{\pi_{\text{ref}}(x)}{\pi_{\theta}(x)} \nabla_{\theta} \log \pi_{\theta}(x) \right]$
- k2估计器有偏，但梯度与真实KL散度梯度一致
- k3估计器无偏，但期望梯度不准
- straight-through (k3+)

前向传播用k3，反向传播用k2

```
forward_score = kl_penalty_forward(logprob, ref_logprob, kl_penalty)
backward_score = 0.5 * (logprob - ref_logprob).square()
return backward_score - backward_score.detach() + forward_score.detach()
```

这样前向传播和反向传播就同时准确了